



Discrete Flow Matching is a Surprisingly Effective Post-training Method to Address Compound Error in Autoregressive Models

Kang Li**
Department of Statistics
University of Oxford
Oxford, United Kingdom
kang.li@stats.ox.ac.uk

Bidipta Sarkar*
Foerster Lab for AI Research
University of Oxford
Oxford, United Kingdom
bidipta.sarkar@eng.ox.ac.uk

Zheng Xiong*
Department of Computer Science
University of Oxford
Oxford, United Kingdom
zheng.xiong@eng.ox.ac.uk

Sascha Frey
Department of Computer Science
University of Oxford
Oxford, United Kingdom
sascha.frey@st-hughs.ox.ac.uk

Zilin Wang
Foerster Lab for AI Research
University of Oxford
Oxford, United Kingdom
zilin.wang@lmh.ox.ac.uk

Frensi Zejnullahu
Department of Statistics
University of Oxford
Oxford, United Kingdom
zejnufre@outlook.com

Alfred Backhouse
Department of Computer Science
University of Oxford
Oxford, United Kingdom
alfred.backhouse@gmail.com

Stefan Zohren
Department of Engineering Science
University of Oxford
Oxford, United Kingdom
stefan.zohren@eng.ox.ac.uk

Anisoara Calinescu
Department of Computer Science
University of Oxford
Oxford, United Kingdom
ani.calinescu@cs.ox.ac.uk

Mihai Cucuringu^{††}
Department of Mathematics
University of California Los Angeles
Los Angeles, USA
Department of Statistics & Oxford
Man Institute of Quantitative Finance
Oxford, United Kingdom
mihai@math.ucla.edu

Jakob Foerster[†]
Foerster Lab for AI Research
University of Oxford
Oxford, United Kingdom
jakob@robots.ox.ac.uk

Abstract

Autoregressive (AR) models suffer from compound error accumulation, where prediction errors propagate and amplify throughout sequence generation. We introduce unified discrete flow matching (DFM) as a post-training framework that eliminates this fundamental limitation of AR models. By converting pretrained AR models into flow-based generators, our approach enables bidirectional context integration and iterative error correction while preserving learned representations. Our framework leverages the pretrained model’s distribution as the starting point for flow matching, reducing the learning burden compared to noise-based approaches. Unlike BERT-style masked models with discrete diffusion that suffer from mode averaging, our unified flow matching framework successfully learns discrete patterns, achieving 99.9% validity on solving mode averaging tasks where traditional methods completely

fail. Critically, our experiments demonstrate that unified DFM completely eliminates compound error: while AR models show a Wasserstein distance that is increasing throughout generation progress, unified DFM maintains a nearly flat error profile throughout the entire sequence. This post-training approach achieves a 48.6% reduction in Wasserstein distance.

Keywords

Unified discrete flow matching, compound error elimination, autoregressive models, efficient inference, post-training.

ACM Reference Format:

Kang Li, Bidipta Sarkar, Zheng Xiong, Sascha Frey, Zilin Wang, Frensi Zejnullahu, Alfred Backhouse, Stefan Zohren, Anisoara Calinescu, Mihai Cucuringu, and Jakob Foerster. 2025. Discrete Flow Matching is a Surprisingly Effective Post-training Method to Address Compound Error in Autoregressive Models. In *6th ACM International Conference on AI in Finance (ICAIF '25)*, November 15–18, 2025, Singapore, Singapore. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3768292.3770442>

*Authors contributed equally to this research.

†Authors advised equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License. ICAIF '25, Singapore, Singapore

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2220-2/25/11

<https://doi.org/10.1145/3768292.3770442>

1 Introduction

Recent advances in generative modelling have shown remarkable success in multimodal tasks, with autoregressive (AR) models dominating the landscape [3, 19]. However, AR models suffer from inherent limitations: strict left-to-right generation constraints, the

inability to revise previously generated tokens, and, crucially, compound error accumulation during sequential generation [16, 17]. In AR models, each prediction conditions on previous outputs, causing errors to propagate and amplify throughout the sequence, degrading generation quality, especially for long sequences.

Discrete Flow Matching (DFM) [4, 10, 14, 22, 27] has emerged as a compelling alternative, enabling bidirectional context integration and iterative refinement during generation. Unlike AR models, DFM can correct errors through its iterative denoising process over the whole sequence. With sequences generated from the pretrained AR model as the starting point and the true data pattern as the goal, DFM can iteratively refine the sequence to the true data pattern.

In this paper, we present the unified DFM as a post-training framework that eliminates compound error from pretrained AR models. Our key contributions are:

- (1) **Post-training Framework:** We open-source a method to convert pretrained AR transformer models into flow-based generators through sample-efficient post-training, preserving learned representations while gaining bidirectional refinement capabilities. Unlike approaches that require separate architectures for different modalities [7, 15, 28], we maintain a unified post-training network shared with the pretrained AR model.
To enable smoother post-training on pretrained models, we introduce a learnable residual connection from the embedding layer to the output, initialized to zeros. This design enables a smooth transition from autoregressive to discrete flow matching modes while preserving learned representations. Furthermore, unlike traditional flow matching approaches that start from random noise, our framework leverages the pretrained model’s learned distribution as the initial point of the flow. That is, the DFM process begins from the Self-Supervised Learning (SSL) distribution ($x_0 \sim p_{\text{SSL}}$) and flows towards the true data distribution ($x_1 \sim p_{\text{data}}$).
- (2) **Learnable Residual Connection:** We introduce a novel learnable residual connection with zero initialization that enables smooth transition from pretrained AR models to flow-based generation. Unlike traditional residual connections, our design includes a learnable projection matrix $\mathbf{P}$ that starts from zero and gradually learns direct input-output mappings.
- (3) **Compound Error Elimination:** By enabling iterative refinement with bidirectional context, our approach removes the compound error inherent in AR generation, improving quality, especially for long sequences.

2 Related Work

2.1 Discrete Generative Models

Discrete generative modelling has evolved from early autoregressive approaches [26] to more sophisticated frameworks. AR models generate tokens sequentially, which limits their ability to perform complex reasoning and self-correction. Recent work has explored alternatives, including:

Mask-Based Diffusion [1, 5, 18, 31]: These models corrupt data by masking tokens and learn to reconstruct them. When predicting multiple masked tokens in parallel without causal dependencies, the model optimises for maximum likelihood estimation (MLE) independently at each position [29], learning the conditional probabilities for a wide range of masked sequences. These have been popular alternatives to diffusion for image generation [5], resulting in strong FID scores while taking less time than AR models since they unmask multiple tokens in parallel. However, unmasked tokens remain fixed during generation, limiting flexibility.

Discrete Flow Matching [4, 13]: Early work on DFM defined continuous-time flows in discrete spaces, enabling smooth transitions between distributions. FUDOKI [27] introduced metric-induced probability paths, which use embedding distances to guide the flow process, pioneering a more semantically meaningful approach to discrete flow matching for multimodal generation.

2.2 Error Accumulation in Generative Models

Compound error (exposure bias) has been a long-standing challenge in sequential generation [2, 20]. During training, models see ground truth contexts, but during inference, they condition on their own predictions, leading to distribution shift and error accumulation. Particularly in high-frequency trading applications dealing with market-by-order data, this can be problematic. Generative models for HFT [16, 17] are showing promising results for synthetic data generation, but suffer from distributional shift. The benchmarking work in [16] identifies this, but does not offer a solution.

There have been previous attempts to mitigate compound error which include; Scheduled sampling [2], Professor forcing [11] and Adversarial training [30]. Our work takes a different approach by adapting the flow dynamics based on predicted uncertainty.

3 Background

3.1 Discrete Flow Matching

3.1.1 Notation and Preliminaries. We use the following notation throughout the paper:

- x^i : denotes the token at position i in a sequence (superscript indicates position index)
- $p_t(x)$: marginal probability distribution at time $t \in [0, 1]$
- $p_t(x|x_0, x_1)$: conditional probability distribution given source x_0 and target x_1
- $p_\theta(x_1|x_t, t)$: learned model predicting target x_1 from corrupted x_t at time t
- $u_t(x)$: marginal probability velocity field
- $u_t^i(x^i, z)$: conditional probability velocity at position i , representing the rate of jumping from current state z to candidate state x^i
- $d(\cdot, \cdot)$: distance function in embedding space
- d_j^i : shorthand for $d(j, x_1^i)$, the distance from token j to target at position i
- β_t : time-dependent corruption schedule with $\beta_0 = 0$ and $\beta_1 = \infty$

Discrete Flow Matching defines a probability path $\{p_t(x)\}_{t \in [0, 1]}$ that interpolates between a source distribution $p_0(x)$ and target distribution $p_1(x)$. The flow is governed by probability velocities

$u_t(x)$ satisfying the Continuity Equation:

$$\frac{\partial p_t(x)}{\partial t} + \nabla \cdot (p_t(x)u_t(x)) = 0, \quad (1)$$

For discrete spaces, the divergence operator takes a specific form. While we use $\nabla \cdot$ for the continuous case above, we denote the discrete divergence as $\nabla_x \cdot$ to emphasise its discrete nature. Given a probability flux $v : \mathcal{D} \times \mathcal{D} \rightarrow \mathbb{R}$, the discrete divergence is:

$$\nabla_x \cdot v = \sum_{z \in \mathcal{D}} [v(z, x) - v(x, z)], \quad (2)$$

where $v(x, z)$ represents the flux from state x to state z .

Our approach builds upon the theoretical foundations established by Discrete Flow Matching [27], which introduces discrete flow paradigms for text and code generation. While both methods utilise probability velocities and corruption processes, our work diverges in several key aspects.

We adopt metric-based probability paths where the transition probability depends on the distance between tokens in the embedding space:

$$p_t(x^i | x_1^i) = \text{softmax}(-\beta_t \cdot d(x^i, x_1^i)), \quad (3)$$

where $d(\cdot, \cdot)$ measures embedding distances and β_t controls corruption strength.

3.2 Mode Collapse and Averaging

Common reasons for generative models failing at distribution matching include “mode collapse” and “mode averaging”. In mode collapse [9], the generative model outputs the most likely mode of data instead of following the true distribution. For instance, in a distribution matching task with only two possible generations, a mode collapsed model may learn to only output the more likely generation, never generating the rarer outcome.

In mode averaging, models generate samples that are blends of multiple valid samples instead of strictly following discrete modes [6, 24]. This is especially problematic for masked-based models, which can encounter mode averaging even with the “ideal” model which perfectly captures the marginal and conditional distributions of all masked tokens, given a masked sample of the true data. This is because independently sampling multiple tokens would result in a different distribution than correctly modeling the joint distribution due to the incorrect independence assumption [8, 12]. For instance, to model a distribution of the constant sequences $000\dots$ and $111\dots$ being generated with equal probability, the model correctly models the marginal 50% probability for both 0 and 1 at each position, but if the first two tokens are generated at the first inference step, each of 00 , 01 , 10 , and 11 are equally likely to be generated, even though only 00 and 11 are valid.

4 Method

4.1 Flow Matching for Post-training

Autoregressive SSL models suffer from compound error accumulation during sequential generation. As we demonstrate experimentally in Section 5 (Figure 1), flow matching eliminates this fundamental limitation through its iterative refinement process with bidirectional context. Unlike AR models, which cannot revise

previous predictions, flow matching can correct errors throughout the entire sequence during its denoising process.

To make this post-training process more efficient, we leverage the pretrained model’s learned distribution as the starting point for flow matching. Instead of starting from random noise, our approach begins from the SSL model’s output distribution:

$$x_0 \sim p_{\text{SSL}}(x), \quad x_1 \sim p_{\text{data}}(x) \quad (4)$$

Here, $p_0 = p_{\text{SSL}}$ represents the source distribution (pretrained model output) and $p_1 = p_{\text{data}}$ represents the target distribution (true data).

This SSL-based initialization provides two key advantages: First, it significantly reduces the learning burden during post-training, as the flow only needs to learn the refinement from SSL predictions to true data, rather than the entire mapping from noise to data. Second, it preserves the meaningful representations already captured during pretraining, allowing the model to build upon existing knowledge rather than learning from scratch.

4.2 Discrete Flow Matching for Time Series

Our approach adapts discrete flow matching to time series forecasting by treating sequences as discrete token trajectories. Building on the metric-induced probability paths pioneered by [23, 27] and the theoretical framework of discrete flow matching [14], we adopt embedding distance-based paths that provide more semantically meaningful transformations than mask-based approaches.

4.2.1 Metric-Induced Probability Paths. For a time series discretized into tokens $\{x^i\}_{i=1}^T \in \mathcal{V}^T$ where $\mathcal{V}$ is the vocabulary, we use the conditional probability paths defined in (3), where $d : \mathcal{V} \times \mathcal{V} \rightarrow \mathbb{R}_{\geq 0}$ measures distance between tokens in the embedding space, and $\beta_t : [0, 1] \rightarrow \mathbb{R}_{\geq 0}$ is a monotonic schedule with $\beta_0 = 0$ and $\beta_1 = \infty$.

This formulation allows semantically similar tokens to have higher transition probabilities, which is particularly important for financial time series where adjacent price levels have similar economic meanings.

4.2.2 Probability Velocity for Discrete Flows. In discrete flow matching, the probability velocity u_t^i describes the rate of transitioning between states at position i . Specifically, $u_t^i(x^i, z)$ represents the rate of jumping from current state z to state x^i , conditioned on the target distribution.

Following the metric-induced path framework, the optimal velocity for our probability path is:

$$u_t^i(x^i, z) = p_t(x^i | x_1^i) \dot{\beta}_t [d(z, x_1^i) - d(x^i, x_1^i)]_+ \quad (5)$$

where $[\cdot]_+ = \max\{\cdot, 0\}$ ensures non-negative rates and $\dot{\beta}_t$ is the time derivative of β_t . The distance reduction term $[d(z, x_1^i) - d(x^i, x_1^i)]_+$ ensures positive rates only when jumping from z to x^i decreases the distance to the target x_1^i . This formulation ensures probability flows monotonically toward the target distribution. In practice, z represents the current token value x_t^i at position i , making the velocity depend on the current state of the sequence during the flow.

4.2.3 Adaptations for Time Series. Unlike the multimodal setting in FUDOKI, our time series application emphasises the following aspects:

- (1) **Temporal Dependencies:** Financial time series exhibit strong temporal correlations. We preserve these by using causal attention masks during the initial AR pretraining stage, then switch to full attention during flow matching to enable global refinement.
- (2) **Token Embeddings:** We use learned embeddings for discretized price levels, where the embedding distance $d(\cdot, \cdot)$ reflects economic similarity between price movements.

4.3 Training Algorithm

4.3.1 Two-Stage Training Strategy. Our training procedure strategically combines autoregressive pretraining with flow matching adaptation:

Table 1: Two-Stage Training Strategy for Discrete Flow Matching

Stage	Configuration
Stage 1: AR Pre-training	Next-token prediction with cross-entropy loss, causal mask for left-to-right generation, 100 epochs, LR=2e-4 with cosine schedule
Stage 2A : Frozen Backbone	Freeze all pretrained parameters, train only skip connection matrix P, switch to full bidirectional mask, LR=1e-4 for new parameters
Stage 2B: Full Fine-tuning	Unfreeze all parameters, differential learning rates: $\alpha_{\text{new}}=1\text{e-}4$, $\alpha_{\text{pretrained}}=5\text{e-}5$, gradient clipping with max norm=1.0

4.3.2 Corruption Process. For each training step, we sample corrupted sequences following the metric-induced probability path. Algorithm 1 details this corruption procedure, where we sample a target sequence x_1 from the data distribution $q(\cdot)$, apply metric-based corruption using the embedding function $e(\cdot) : \mathcal{V} \rightarrow \mathbb{R}^D$ that maps tokens to D -dimensional vectors, and compute normalized distances $\tilde{d}_j^i = d_j^i / \max_k d_k^i$ to ensure stable probability computations. The algorithm returns both the corrupted sequence x_t and the original target x_1 for training.

4.3.3 Schedule Design. We employ a cosine schedule for β_t :

$$\beta_t = \beta_{\max} \cdot \frac{1 - \cos(\pi t)}{2} \quad (6)$$

where $\beta_{\max} = 10$ ensures near-deterministic behavior at $t = 1$.

4.3.4 Training Objective. The training objective minimizes the expected cross-entropy between predicted and true token distributions:

$$\mathcal{L}_{\text{CE}}(\theta) = \mathbb{E}_{t \sim U[0,1], x_1 \sim q(\cdot), x_t \sim p_t(\cdot | x_1)} \left[- \sum_{i=1}^T \log p_{1|t}^{\theta}(x_1^i | x_t) \right] \quad (7)$$

Algorithm 1 Training Data Corruption

- 1: Sample target sequence $x_1 \sim q(\cdot)$ from data
 - 2: Sample time $t \sim \text{Uniform}[0, 1]$
 - 3: Compute β_t using cosine schedule
 - 4: **for** each position $i = 1$ to T **do**
 - 5: Compute embedding distances: $d_j^i = d(j, x_1^i) = \|e(j) - e(x_1^i)\|_2$ for all $j \in \mathcal{V}$
 - 6: Normalize: $\tilde{d}_j^i = d_j^i / \max_k d_k^i$
 - 7: Compute probabilities: $p_t(j | x_1^i) = \text{softmax}(-\beta_t \cdot \tilde{d}_j^i)$
 - 8: Sample $x_t^i \sim p_t(\cdot | x_1^i)$
 - 9: **end for**
 - 10: **return** Corrupted sequence x_t , target x_1 , time t
-

where $p_{1|t}^{\theta}(\cdot | x_t)$ is the model’s predicted categorical distribution parameterized by θ . This cross-entropy objective directly trains the model to learn the posterior distribution $p_{1|t}(x_1^i | x_t)$. During inference, we use this learned posterior to sample $\hat{x}_1^i$, which then allows us to compute $p_t(x^i | \hat{x}_1^i)$ needed in the velocity formula (see Algorithm 3, Line 13). Thus, minimizing cross-entropy loss provides exactly the component required for implementing the kinetic optimal velocities during the flow matching process.

4.3.5 Complete Training Algorithm. Algorithm 2 presents our complete two-stage training procedure that transforms a pretrained AR model into a flow matching generator. The algorithm first performs standard autoregressive pretraining with causal masking, where $\mathcal{L}_{\text{AR}} = - \sum_i \log p(x_i | x_{<i})$ denotes the next-token prediction loss. In Stage 2, we adapt the model for flow matching through a carefully designed post-training process: Phase 2A freezes the backbone and trains only the residual connection, while Phase 2B fine-tunes all parameters with differential learning rates to preserve pretrained knowledge while learning flow dynamics.

4.4 Model Architecture

4.4.1 Overview. Our architecture adapts the flow matching framework to time series by building on pretrained autoregressive models. We make minimal but crucial modifications to enable discrete flow matching while preserving the learned representations from AR pretraining.

Core Components. The model architecture consists of three main components: the transformer backbone, token embeddings, and an evolving attention mechanism. The attention mechanism undergoes a critical evolution between training stages. In Stage 1 (AR Pretraining), we employ a causal attention mask that enforces left-to-right dependencies. In Stage 2 (Flow Matching), we switch to a full attention mask enabling bidirectional context, which is crucial for global pattern recognition, error correction during denoising, and capturing long-range dependencies.

4.4.2 Key Components. Learnable Residual Connection: To enable smooth adaptation from pretrained AR models, we introduce a learnable residual connection from embeddings to output. Let $h_{\text{embed}} \in \mathbb{R}^{T \times 64}$ denote the input embeddings, $h_{\text{AR}} \in \mathbb{R}^{T \times 128}$ denote the transformer output from the pretrained AR model, and

Algorithm 2 Discrete Flow Matching Training with Post-training

Require: Pretrained AR model f_{AR} , dataset $\mathcal{D}$, epochs E_1, E_2, E_3

Ensure: Trained flow matching model f_θ

```

1: Stage 1: AR Pretraining
2: for epoch = 1 to  $E_1$  do
3:   for batch  $x \in \mathcal{D}$  do
4:     Compute AR loss:  $\mathcal{L}_{AR} = -\sum_i \log p(x_i|x_{<i})$ 
5:     Update  $f_{AR}$  with Adam optimizer
6:   end for
7: end for
8:
9: Stage 2: Flow Matching Adaptation
10: Initialize skip connection  $P \leftarrow 0$ 
11:  $f_\theta \leftarrow f_{AR}$  with full attention mask
12:
13: Phase 2A: Frozen Backbone
14: for epoch = 1 to  $E_2$  do
15:   for batch  $x_1 \in \mathcal{D}$  do
16:     Sample  $t \sim U[0, 1]$ 
17:     Compute  $\beta_t = \beta_{\max} \cdot (1 - \cos(\pi t))/2$ 
18:     Corrupt:  $x_t \sim p_t(\cdot|x_1)$  using embedding distances
19:     Forward pass:  $\hat{p} = f_\theta(x_t)$ 
20:     Compute loss:  $\mathcal{L}_{CE}$ 
21:     Update only  $P$ 
22:   end for
23: end for
24:
25: Phase 2B: Full Fine-tuning
26: for epoch =  $E_2 + 1$  to  $E_3$  do
27:   for batch  $x_1 \in \mathcal{D}$  do
28:     Sample  $t$ , corrupt to get  $x_t$  (same as Phase 2A)
29:     Forward pass:  $\hat{p} = f_\theta(x_t)$ 
30:     Compute loss  $\mathcal{L}_{CE}(\theta)$ 
31:     Update all parameters with differential LR:
32:        $\theta_{\text{new}} \leftarrow \theta_{\text{new}} - \alpha_{\text{new}} \nabla \mathcal{L}$ 
33:        $\theta_{\text{pretrained}} \leftarrow \theta_{\text{pretrained}} - \alpha_{\text{pretrained}} \nabla \mathcal{L}$ 
34:   end for
35: end for
36: return  $f_\theta$ 

```

$h_{\text{out}} \in \mathbb{R}^{T \times 128}$ denote the final output representation. The residual connection is formulated as:

$$h_{\text{out}} = h_{AR} + \text{LayerNorm}(h_{\text{embed}}P) \quad (8)$$

where:

- $P \in \mathbb{R}^{64 \times 128}$: Learnable projection matrix that is initialized to zero and gradually learned during post-training
- LayerNorm: Stabilizes the residual branch during training
- Initialization: Zero-initialized for smooth transition from pretrained weights

This design ensures the model starts from the exact pretrained solution when P is zero and gradually learns direct input-output mappings for flow matching. The zero initialization is crucial as it allows the model to initially produce the same outputs as the

pretrained AR model, then progressively adapt to the flow matching objective.

Smooth Transition from AR to DFM: The learnable residual connection enables a smooth transition from autoregressive (AR) to discrete flow matching (DFM) modes. During the early stages of post-training, when P is near zero, the model primarily relies on the pretrained AR representations. As training progresses and P learns meaningful patterns, the model gradually incorporates direct input-to-output mappings that are essential for bidirectional flow matching. This smooth transition preserves the valuable knowledge from AR pretraining while enabling the model to overcome the limitations of left-to-right generation.

Output Head: The model uses a single output head that projects the final hidden states to vocabulary logits. Let $h_{\text{final}} = h_{\text{out}} \in \mathbb{R}^{T \times 128}$ denote the output from the residual connection. The projection to vocabulary space is:

$$\text{logits} = \text{Linear}_{128 \rightarrow |\mathcal{V}|}(h_{\text{final}}) \quad (9)$$

This straightforward design maintains compatibility with the pretrained AR model while enabling flow matching capabilities.

4.4.3 Key Design Choices. Shifting Operation: To preserve AR capabilities during adaptation, we apply output shifting, where $x_{<i}$ denotes tokens before position i and $x_{\geq i}^{\text{corrupted}}$ denotes corrupted tokens from position i onwards:

$$\text{logits}[i] = f_\theta(x_{<i}, x_{\geq i}^{\text{corrupted}})[i-1] \quad (10)$$

This allows the model to leverage its next-token prediction training even when using full attention.

Design Rationale. The learnable residual connection design serves multiple purposes:

- (1) **Preserves Input Information:** Direct path from embeddings to output maintains access to raw token information
- (2) **Minimal Parameters:** Adds only D^2 parameters (16,384 for $D = 128$)
- (3) **Smooth Transition:** Zero initialization ensures the model starts from the pretrained solution
- (4) **Gradient Flow:** Residual structure prevents vanishing gradients during post-training

4.5 Advantages over Mask-Based Approaches

Our metric-induced flow matching offers several advantages over mask-based discrete diffusion:

- (1) **Continuous Refinement:** Tokens can be updated throughout the denoising process, allowing error correction. Mask-based methods [21, 25] typically cannot modify tokens once unmasked.
- (2) **Semantic Transitions:** The embedding-based distance metric enables semantically meaningful transitions between price levels, crucial for financial data.
- (3) **Efficient Sampling:** Starting from SSL distribution reduces the number of steps needed for high-quality generation.

4.6 Inference Algorithm

Our key innovation is combining discrete flow matching with SSL-based initialization. Algorithm 3 presents the complete inference

algorithm that enables efficient generation starting from the pre-trained model’s distribution.

Algorithm 3 Discrete Flow Matching Inference with SSL Initialization

Require: Trained model f_θ , number of steps N , sequence length T , SSL model (optional)

Ensure: Generated sequence x_1

```

1: Initialization:
2: if SSL model provided then
3:   Initialize  $x_t$  from SSL model output distribution
4: else
5:   Initialize  $x_t$  uniformly:  $x_t^i \sim \text{Uniform}(\mathcal{V})$  for  $i = 1, \dots, T$ 
6: end if
7: Set initial time  $t = 0$ 
8: Set step size  $\Delta t = 1/N$ 
9: while  $t < N$  do
10:  Step 1: Model Prediction
11:  Forward pass:  $\hat{p}_{1|t} = f_\theta(x_t, t)$ 
12:  Sample predictions:  $\hat{x}_1^i \sim \hat{p}_{1|t}(\cdot|x_t)$  for all  $i$ 
13:
14:  Step 2: Discrete Flow Update
15:  for each position  $i = 1$  to  $T$  do
16:    Compute velocity rates:
17:     $u_t^i(j, x_t^i) = p_t(j|\hat{x}_1^i) \cdot \hat{p}_t \cdot [d(x_t^i, \hat{x}_1^i) - d(j, \hat{x}_1^i)]_+$ 
18:    Compute total rate:  $\lambda^i = \sum_{j \neq x_t^i} u_t^i(j, x_t^i)$ 
19:
20:    Sample transition:  $Z^i \sim \text{Uniform}[0, 1]$ 
21:    if  $Z^i \leq 1 - \exp(-\Delta t \cdot \lambda^i)$  then
22:      Update token: sample  $x_{t+\Delta t}^i$  from
23:       $P(j) = \frac{u_t^i(j, x_t^i)}{\lambda^i}$  for  $j \neq x_t^i$ 
24:    else
25:      Keep token:  $x_{t+\Delta t}^i = x_t^i$ 
26:    end if
27:  end for
28:
29:  Step 3: Time Update
30:   $t \leftarrow t + \Delta t$ 
31: end while
32: return  $x_1$ 

```

The inference procedure in Algorithm 3 implements a discrete-time approximation of the continuous flow process. Starting from either SSL predictions or random initialization, the algorithm iteratively refines the sequence through N denoising steps. At each step, it computes velocity rates based on the learned posterior and the metric-induced probability path, then uses a stochastic update rule to determine which tokens to modify. The SSL initialization leverages pretrained representations to provide a meaningful starting point. It can reduce the number of steps required for high-quality generation compared to random initialization.

The SSL initialization provides several important benefits. First, leveraging pretrained representations ensures **quality** since the initial distribution is already meaningful, leading to better final generation quality. Second, the algorithm offers **flexibility** as it can work with or without SSL initialization, allowing users to choose

based on their requirements. Finally, unlike mask-based methods, our approach enables **continuous refinement** where tokens can be updated throughout the process, allowing the model to correct early mistakes.

5 Experiments

Figure 1 demonstrates the fundamental difference in error dynamics between SSL pre-training and DFM post-training. Three key observations emerge: (1) **Compound error elimination:** The SSL model (left, blue) exhibits clear compound error accumulation, with Wasserstein Distance increasing from approximately 3 to over 20 as generation progresses. In contrast, DFM (right, red) maintains a nearly flat error profile throughout the sequence, validating our theoretical analysis that bidirectional refinement prevents error accumulation. (2) **Overall quality improvement:** DFM achieves substantially lower average Wasserstein Distance compared to SSL across all time steps, demonstrating that post-training successfully improves generation quality. At epoch 9, DFM maintains distances around 7-10 while SSL reaches 20+. (3) **Training convergence:** Comparing epochs 9 and 41 reveals algorithm convergence—DFM at epoch 41 (bottom right) shows both lower mean error (approximately 3-4) and reduced variance compared to epoch 9, indicating stable optimization of the flow-matching objective.

5.1 Experimental Setup

Model Architecture:

- Base: 3-layer transformer with 128d hidden, 4 attention heads
- Skip connection: Single 128×128 matrix (16,384 parameters)
- Total parameters: 0.8M base + 16K skip connection

Training Details:

- SSL pretraining: 100 epochs, LR=2e-4
- DFM post-training: 100 epochs, skip connection LR=1e-4, model LR=5e-5
- Freeze epochs: 15
- Batch size: 10,000

5.2 Compound Error Experiments

Datasets. We evaluate our approach on the Electricity Consumption dataset, which contains electricity load readings from a single household located in Sceaux, France, collected over approximately four years (December 2006 - November 2010). The data represents real-world energy consumption patterns with natural temporal dependencies, using minute-averaged active power measurements in kilowatts. For our experiments, we use sequences of length 102 for both training and evaluation, with a vocabulary size of 481 discrete bins after uniform binning and removal of missing values. The dataset is split into 10,000 training samples, 5,000 validation samples, and 5,000 test samples, with results reported across 30 independent random seeds to ensure statistical reliability.

Distribution Matching Analysis. The distribution matching metrics reveal how well each model captures the underlying data distribution. The improvement of Wasserstein Distance (WD) by 48.6% indicates the average distance between the generated and true

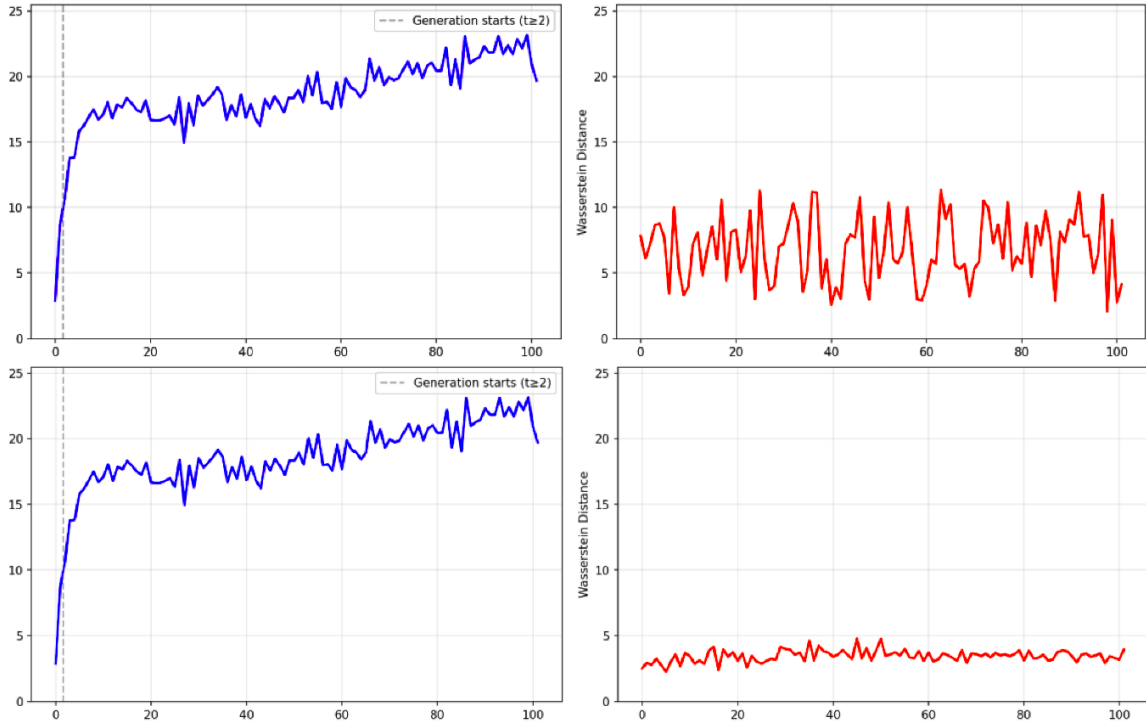


Figure 1: Wasserstein Distance over generation time steps. Left column (blue): SSL pre-training exhibits compound error accumulation. Right column (red): DFM post-training maintains stable error rates. Top row shows results at epoch 9, bottom row at epoch 41. The vertical dashed line indicates where generation begins ($t=2$).

Table 2: Distribution Matching Improvement: DFM Post-training over AR Pretraining

Metric	Improvement (%)
Wasserstein Distance (WD)	48.6%
KL Divergence	64.6%
KL-2gram	-8.0%
KL-3gram	7.3%

distributions, suggesting substantial improvement after DFM post-training. The KL divergence of 64.6% measures the overall distributional mismatch. More importantly, the KL-2gram and KL-3gram metrics specifically evaluate temporal dependencies across time series steps. The negative KL-2gram value (-8.0%) suggests that the DFM model slightly overemphasizes pairwise temporal patterns compared to the data, while the positive KL-3gram (7.3%) indicates mild underrepresentation of three-step dependencies. These n-gram KL divergences are crucial for time series modeling as they capture whether the model correctly learns the temporal structure beyond simple token distributions. The relatively small magnitudes of these n-gram metrics demonstrate that DFM successfully preserves temporal relationships while eliminating compound error.

5.3 Mode Averaging Experiments

5.3.1 Problem Description. Mode averaging represents a fundamental challenge in discrete generative modeling, particularly for deterministic pattern learning. This problem manifests when traditional Maximum Likelihood Estimation (MLE) methods encounter data distributions containing multiple discrete modes. In our experiments, we focus on alternating binary patterns—Pattern 1 (01010101...) and Pattern 2 (10101010...)—which provide a clear test case for this phenomenon.

Under MLE training, models learn to predict the average probability distribution across all training examples. For alternating patterns, this results in 50% probability predictions for both 0 and 1 at each position. While statistically correct in terms of marginal probabilities, this averaging catastrophically fails to capture the structural constraints of the data, generating invalid sequences such as 0000... or 1111... that never appear in the training distribution.

5.3.2 Experimental Setup. We evaluate DFM’s ability to overcome mode averaging using synthetic alternating pattern datasets with the following parameters:

- **Sequence Lengths:** 16, 32, 64, and 128 tokens
- **Training Samples:** 10,000 sequences per length
- **Test Samples:** 1,000 sequences for generation evaluation
- **Model Configuration:**
 - 3-layer transformer with 128-dimensional hidden states
 - 4 attention heads, 64-dimensional embeddings

- Single-layer skip connection with 16,384 parameters
- **Evaluation Metrics:**
 - Perfect sequence rate: Percentage of generated sequences matching valid patterns
 - Error count: Number of positions deviating from alternating pattern
 - Pattern distribution: Balance between Pattern 1 and Pattern 2 generation

Table 3: Error statistics comparison between MLE and DFM

Sequence Length	Average Errors		Median Errors	
	MLE	DFM	MLE	DFM
16 tokens	7.55	0.00	8	0
32 tokens	15.52	0.00	15	0
64 tokens	31.71	0.00	32	0
128 tokens	63.60	0.00	63	0

5.3.3 Quantitative Results. Tables 3 and 4 provide detailed error analysis. MLE models exhibit errors proportional to sequence length, with approximately 50% of positions incorrect—consistent with random 50/50 predictions. In contrast, DFM achieves zero errors across all percentiles and sequence lengths, demonstrating robust pattern learning.

6 Conclusion

We presented a post-training framework for Discrete Flow Matching that addresses both compound error accumulation and mode averaging problems. Our experiments demonstrate that DFM completely eliminates compound error: while AR models show Wasserstein Distance increasing throughout generation progresses, DFM maintains a nearly flat error profile throughout the entire sequence. On mode averaging tasks with alternating binary patterns, DFM achieves 100.0% validity in generating structurally correct sequences, while traditional MLE methods completely fail with 0% validity. The post-training approach achieves 48.6% reduction in Wasserstein Distance and 64.6% improvement in KL divergence compared to pretrained AR models. By leveraging the pretrained model’s distribution as the starting point for flow matching and introducing a learnable residual connection, our unified framework offers a practical path to solve the compound error in discrete generation.

Acknowledgments

AC acknowledges funding from a UKRI AI World Leading Researcher Fellowship (grant EP/W002949/1). AC and JF acknowledge funding from a JPMC Faculty Research Award. We thank Peer Nagy from the OMI, University of Oxford, and Tao Hu from the Ommer Lab, Ludwig Maximilian University of Munich, for valuable feedback and discussions.

References

- [1] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. 2021. Structured Denoising Diffusion Models in Discrete State-Spaces. In *Advances in Neural Information Processing Systems*, A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan (Eds.). <https://openreview.net/forum?id=h7-XixPCAL>
- [2] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. 2015. Scheduled sampling for sequence prediction with recurrent Neural networks. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1* (Montreal, Canada) (*NIPS’15*). MIT Press, Cambridge, MA, USA, 1171–1179.
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc967418bfb8ac142f64a-Paper.pdf
- [4] Andrew Campbell, Jason Yim, Regina Barzilay, Tom Rainforth, and Tommi Jaakkola. 2024. Generative flows on discrete state-spaces: enabling multimodal flows with applications to protein co-design. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (*ICML’24*). JMLR.org, Article 213, 60 pages.
- [5] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. 2022. MaskGIT: Masked Generative Image Transformer. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 11305–11315. doi:10.1109/CVPR52688.2022.01103
- [6] Tong Che, Yanran Li, Athul Jacob, Yoshua Bengio, and Wenjie Li. 2017. Mode Regularized Generative Adversarial Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=HJkY35le>
- [7] Xiaokang Chen, Zhiyu Wu, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, and Chong Ruan. 2025. Janus-Pro: Unified Multimodal Understanding and Generation with Data and Model Scaling. arXiv:2501.17811 [cs.AI] <https://arxiv.org/abs/2501.17811>
- [8] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. 2019. Mask-Predict: Parallel Decoding of Conditional Masked Language Models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 6112–6121. doi:10.18653/v1/D19-1633
- [9] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative Adversarial Networks. arXiv:1406.2661 [stat.ML] <https://arxiv.org/abs/1406.2661>
- [10] Kacper Papisniak, Peter Potaptchik, Teodora Reu, Leo Zhang, Alexander Tong, Michael Bronstein, Avishek Joey Bose, and Francesco Di Giovanni. 2024. Metric Flow Matching for Smooth Interpolations on the Data Manifold. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 135011–135042. https://proceedings.neurips.cc/paper_files/paper/2024/file/f381114cf5aba4e4552869863deaaa7-Paper-Conference.pdf
- [11] Alex M Lamb, Anirudh Goyal ALIAS PARTH GOYAL, Ying Zhang, Saizheng Zhang, Aaron C Courville, and Yoshua Bengio. 2016. Professor Forcing: A New Algorithm for Training Recurrent Networks. In *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (Eds.), Vol. 29. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2016/file/16026d60ff9b54410b3435b403afd226-Paper.pdf
- [12] Yuchen Li, Alexandre Kirchmeyer, Aashay Mehta, Yilong Qin, Boris Dadachev, Kishore Papineni, Sanjiv Kumar, and Andrej Risteski. 2024. Promises and Pitfalls of Generative Masked Language Modeling: Theoretical Framework and Practical Guidelines. arXiv:2407.21046 [cs.CL] <https://arxiv.org/abs/2407.21046>
- [13] Yaron Lipman et al. 2023. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747* (2023).
- [14] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. 2023. Flow Matching for Generative Modeling. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=PqvMRDCJT9t>
- [15] Yiyang Ma, Xingchao Liu, Xiaokang Chen, Wen Liu, Chengyue Wu, Zhiyu Wu, Zizheng Pan, Zhenda Xie, Haowei Zhang, Xingkai yu, Liang Zhao, Yisong Wang, Jiaying Liu, and Chong Ruan. 2025. JanusFlow: Harmonizing Autoregression and Rectified Flow for Unified Multimodal Understanding and Generation. arXiv:2411.07975 [cs.CV] <https://arxiv.org/abs/2411.07975>
- [16] Peer Nagy, Sascha Frey, Kang Li, Bidipta Sarkar, Svitlana Vyetrenko, Stefan Zohren, Ani Calinescu, and Jakob Foerster. 2025. LOB-Bench: Benchmarking Generative AI for Finance – an Application to Limit Order Book Data. arXiv:2502.09172 [cs.LG] <https://arxiv.org/abs/2502.09172>
- [17] Peer Nagy, Sascha Frey, Silvia Saporita, Kang Li, Anisoara Calinescu, Stefan Zohren, and Jakob Foerster. 2023. Generative AI for End-to-End Limit Order Book Modelling: A Token-Level Autoregressive Generative Model of Message Flow Using a Deep State Space Network. arXiv:2309.00638 [q-fin.TR] <https://arxiv.org/abs/2309.00638>

Table 4: Error percentile distribution: MLE vs DFM across sequence lengths

Sequence Length	MLE Percentiles					DFM Percentiles				
	50th	75th	90th	95th	99th	50th	75th	90th	95th	99th
16 tokens	8	9	10	11	12	0	0	0	0	0
32 tokens	15	18	19	20	22	0	0	0	0	0
64 tokens	32	34	37	38	42	0	0	0	0	0
128 tokens	63	68	71	73	77	0	0	0	0	0

- [18] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large Language Diffusion Models. [arXiv:2502.09992](https://arxiv.org/abs/2502.09992) [cs.CL] <https://arxiv.org/abs/2502.09992>
- [19] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. 2021. Zero-Shot Text-to-Image Generation. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 8821–8831. <https://proceedings.mlr.press/v139/ramesh21a.html>
- [20] Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. 2016. Sequence Level Training with Recurrent Neural Networks. [arXiv:1511.06732](https://arxiv.org/abs/1511.06732) [cs.LG] <https://arxiv.org/abs/1511.06732>
- [21] Subham Sahoo et al. 2024. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524* (2024).
- [22] Subham Sekhar Sahoo, Justin Deschenaux, Aaron Gokaslan, Guanghan Wang, Justin Chiu, and Volodymyr Kuleshov. 2025. The Diffusion Duality. [arXiv:2506.10892](https://arxiv.org/abs/2506.10892) [cs.LG] <https://arxiv.org/abs/2506.10892>
- [23] Neta Shaul, Itai Gat, Marton Havasi, Daniel Severo, Anuroop Sriram, Peter Hold-errieth, Brian Karrer, Yaron Lipman, and Ricky T. Q. Chen. 2025. Flow Matching with General Discrete Paths: A Kinetic-Optimal Perspective. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=tcvMzR2NrP>
- [24] Akash Srivastava, Lazar Valkov, Chris Russell, Michael U. Gutmann, and Charles Sutton. 2017. VEEGAN: Reducing Mode Collapse in GANs using Implicit Variational Learning. [arXiv:1705.07761](https://arxiv.org/abs/1705.07761) [stat.ML] <https://arxiv.org/abs/1705.07761>
- [25] Zhicong Tang, Tiankai Hang, Shuyang Gu, Dong Chen, and Baining Guo. 2024. Simplified Diffusion Schrödinger Bridge. [arXiv:2403.14623](https://arxiv.org/abs/2403.14623) [cs.LG] <https://arxiv.org/abs/2403.14623>
- [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [27] Jin Wang, Yao Lai, Aoxue Li, Shifeng Zhang, Jiacheng Sun, Ning Kang, Chengyue Wu, Zhenguo Li, and Ping Luo. 2025. FUDOKI: Discrete Flow-based Unified Understanding and Generation via Kinetic-Optimal Velocities. (2025). [arXiv:2505.20147](https://arxiv.org/abs/2505.20147) [cs.CV] <https://arxiv.org/abs/2505.20147>
- [28] Chengyue Wu, Xiaokang Chen, Zhiyu Wu, Yiyang Ma, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, Chong Ruan, et al. 2024. Janus: Decoupling visual encoding for unified multimodal understanding and generation. *arXiv preprint arXiv:2410.13848* (2024).
- [29] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. *Advances in neural information processing systems* 32 (2019).
- [30] Lantao Yu, Weinan Zhang, Jun Wang, and Yong Yu. 2017. SeqGAN: Sequence Generative Adversarial Nets with Policy Gradient. [arXiv:1609.05473](https://arxiv.org/abs/1609.05473) [cs.LG] <https://arxiv.org/abs/1609.05473>
- [31] Lin Zheng, Jianbo Yuan, Lei Yu, and Lingpeng Kong. 2024. A Reparameterized Discrete Diffusion Model for Text Generation. [arXiv:2302.05737](https://arxiv.org/abs/2302.05737) [cs.CL] <https://arxiv.org/abs/2302.05737>